KOÇ UNIVERSITY

COMP 350
Introduction to DevOps

Lecture 5
Storage Media & File Systems

Hakan Ayral

Storage Media

- In previous lectures we've looked at manipulating data at the **file level**.
- In this lecture, we will consider data at the **device level**.
- Linux has amazing capabilities for handling storage devices, whether **physical storage** such as hard disks and ssd, **network storage**, or **virtual storage** devices like RAID (redundant array of independent disks) and LVM (logical volume manager).

Storage Media

We will look at the following commands:

- **mount**—Mount a filesystem.
- **umount**—Unmount a filesystem.
- **fdisk**—Partition table manipulator.
- **fsck**—Check and repair a filesystem.
- **fdformat**—Format a floppy disk.
- **mkfs**—Create a filesystem.
- **dd**—Write block-oriented data directly to a device.
- **genisoimage** (**mkisofs**)—Create an ISO 9660 image file.
- **wodim** (cdrecord)—Write data to optical storage media.
- **md5sum**—Calculate an MD5 checksum.

Mounting and Unmounting Storage Devices

- Recent advances in the Linux desktop have made storage device management extremely easy for desktop users.
  - For the most part, we attach a device to our system and it just works.
  - Back in the old days (say, 2004), this stuff had to be done manually.
- On non-desktop systems (i.e., servers) this is still a largely manual procedure, because servers often have extreme storage needs and complex configuration requirements.
- The first step in managing a storage device is attaching the device to the filesystem tree.
- This process, called **mounting**, allows the device to participate with the operating system.
- Unix-like operating systems, like Linux, maintain a single filesystem tree with devices attached at various points.
  - This contrasts with other operating systems such as MS-DOS and Windows that maintain separate trees for each device (for example C:\, D:\, etc.).

Mounting and Unmounting Storage Devices

- A file named **/etc/fstab** lists the devices (typically hard disk partitions) that are to be mounted at boot time.
- Here is an example **/etc/fstab** file from a Fedora 7 system:
  - Most of the filesystems listed in this example file are virtual and are not applicable to our discussion.
  - For our purposes, the interesting ones are the first three.

```
LABEL=/12           /                       ext3    defaults        1 1
LABEL=/home         /home                   ext3    defaults        1 2
LABEL=/boot         /boot                   ext3    defaults        1 2
tmpfs               /dev/shm                tmpfs   defaults        0 0
devpts              /dev/pts                devpts  gid=5,mode=620  0 0
sysfs               /sys                    sysfs   defaults        0 0
proc                /proc                   proc    defaults        0 0
LABEL=SWAP-sda3     swap                    swap    defaults        0 0
```

Mounting and Unmounting Storage Devices

```
LABEL=/12       /          ext3    defaults        1 1
LABEL=/home     /home      ext3    defaults        1 2
LABEL=/boot     /boot      ext3    defaults        1 2
```

- These are the hard disk partitions.
- Each line of the file consists of six fields, as shown in Table 15-1.

Table 15-1: */etc/fstab* Fields

Field	Contents	Description
1	Device	Traditionally, this field contains the actual name of a device file associated with the physical device, such as */dev/hda1* (the first partition of the master device on the first IDE channel). But with today's computers, which have many devices that are hot pluggable (like USB drives), many modern Linux distributions associate a device with a text label instead. This label (which is added to the storage medium when it is formatted) is read by the operating system when the device is attached to the system. That way, no matter which device file is assigned to the actual physical device, it can still be correctly identified.
2	Mount point	The directory where the device is attached to the filesystem tree
3	Filesystem type	Linux allows many filesystem types to be mounted. Most native Linux filesystems are ext3, but many others are supported, such as FAT16 (`msdos`), FAT32 (`vfat`), NTFS (`ntfs`), CD-ROM (`iso9660`), etc.
4	Options	Filesystems can be mounted with various options. It is possible, for example, to mount filesystems as read only or to prevent any programs from being executed from them (a useful security feature for removable media).
5	Frequency	A single number that specifies if and when a filesystem is to be backed up with the `dump` command
6	Order	A single number that specifies in what order filesystems should be checked with the `fsck` command

Viewing a List of Mounted Filesystems

- The mount command is used to mount filesystems.
  - Entering the command without arguments will display a list of the filesystems currently mounted:
- The format of the listing is

device **on** *mount_point* **type** *filesystem_type* (*options*)

- First line shows that device */dev/sda2* is mounted as the root filesystem, is of type ext3, and is both readable and writable (the option rw).
- This listing also has two interesting entries at the bottom:
  - The next-to-last entry shows a 2-gigabyte SD memory card in a card reader mounted at */media/disk*,
  - and the last entry is a network drive mounted at */misc/musicbox*.

```
[me@linuxbox ~]$ mount
/dev/sda2 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
/dev/sda5 on /home type ext3 (rw)
/dev/sda1 on /boot type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
none on /proc/sys/fs/binfmt_misc type binfmt_misc (rw)
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw)
fusectl on /sys/fs/fuse/connections type fusectl (rw)
/dev/sdd1 on /media/disk type vfat (rw,nosuid,nodev,noatime,
uhelper=hal,uid=500,utf8,shortname=lower)
twin4:/musicbox on /misc/musicbox type nfs4 (rw,addr=192.168.1.4)
```

Mounting a File System

- For our first experiment, we will work with a CD-ROM.
  - First, let's look at a system before a CD-ROM is inserted:

```
[me@linuxbox ~]$ mount
/dev/mapper/VolGroup00-LogVol00 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
/dev/hda1 on /boot type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
none on /proc/sys/fs/binfmt_misc type binfmt_misc (rw)
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw)
```

- This listing is from a CentOS 5 system that is using LVM to create its root filesystem.
- Like many modern Linux distributions, this system will attempt to automatically mount the CD-ROM after insertion.
- After we insert the disc, we see the following:
  - At the end of the listing, we see that the CD-ROM (which is device ***/dev/hdc*** on this system) has been mounted on ***/media/live-1.0.10-8*** and is type iso9660 (a CDROM).

```
[me@linuxbox ~]$ mount
/dev/mapper/VolGroup00-LogVol00 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
/dev/hda1 on /boot type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
none on /proc/sys/fs/binfmt_misc type binfmt_misc (rw)
sunrpc on /var/lib/nfs/rpc_pipefs type rpc_pipefs (rw)
/dev/hdc on /media/live-1.0.10-8 type iso9660 (ro,noexec,nosuid,nodev,uid=500)
```

Unmounting a File System

- Now that we have the device name of the CD-ROM drive, let's unmount the disc and remount it at another location in the filesystem tree.
- To do this, we become the superuser and unmount the disc with the **umount** command:

```
[me@linuxbox ~]$ su -
Password:
[root@linuxbox ~]# umount /dev/hdc
```

- The next step is to create a new **mount point** for the disc.
  - A ***mount point*** is simply a directory somewhere on the filesystem tree. Nothing special about it.

- It doesn't even have to be an empty directory, though if you mount a device on a non-empty directory, you will not be able to see the directory's previous contents until you unmount the device.

```
[root@linuxbox ~]# mkdir /mnt/cdrom
```

Mounting a File System

- Finally, we mount the CD-ROM at the new mount point.
  - The **-t** option is used to specify the filesystem type:

```
[root@linuxbox ~]# mount -t iso9660 /dev/hdc /mnt/cdrom
```

- Afterward, we can examine the contents of the CD-ROM via the new mount point:

```
[root@linuxbox ~]# cd /mnt/cdrom
[root@linuxbox cdrom]# ls
```

- Notice what happens when we try to unmount the CD-ROM:

```
[root@linuxbox cdrom]# umount /dev/hdc
umount: /mnt/cdrom: device is busy
```

- Why is this? We cannot unmount a device if the device is being used by someone or some process.
  - In this case, we changed our working directory to the mount point for the CD-ROM, which causes the device to be busy.
  - We can solve this by changing the working directory to something other than the mount point:
  - Now the device unmounts successfully.

```
[root@linuxbox cdrom]# cd
[root@linuxbox ~]# umount /dev/hdc
```

Listing the supported filesystems

- The virtual file **/proc/filesystems** shows the file system drivers currently loaded by the kernel.
- Even though it is not a real file, you can look at its contents as if it is a regular text file.
- You can install new file system drivers to support other file systems too.

```
bob@ubuntu:~$ cat /proc/filesystems
nodev   sysfs
nodev   rootfs
nodev   bdev
nodev   proc
nodev   cgroup
nodev   cpuset
nodev   tmpfs
nodev   devtmpfs
nodev   debugfs
nodev   securityfs
nodev   sockfs
nodev   pipefs
nodev   anon_inodefs
nodev   devpts
        ext3
        ext4
nodev   ramfs
nodev   hugetlbfs
        vfat
nodev   ecryptfs
        fuseblk
nodev   fuse
nodev   fusectl
nodev   pstore
nodev   mqueue
```

Linux Filesystem Comparison

Filesystem	EXT4	XFS	Btrfs	ZFS	ReiserFS
Creation Timestamp	✗	✓	✓	✓	✗
Last Read Timestamp	✓	✓	✓	✓	✓
Access Control List	✓	✓	✓	✓	✓
Metadata Checksum	✓	✓	✓	✓	✗
Extended Attributes	✓	✓	✓	✓	✓
Hard/Soft Links	✓	✓	✓	✓	✓
File Change Log	✗	✓	✗	✗	✗
Snapshot Support	✗	✗	✓	✓	✗
Encryption	✓	✗	✗	✓	✗
Deduplication	✗	✓	✓	✓	✗
Data Checksum	✗	✗	✓	✓	✗
Persistent Cache	✗	✗	✗	✓	✗
Compression	✗	✗	✓	✓	✗
Online Grow	✓	✓	✓	✓	✓
Offline Shrink	✓	✗	✓	✗	✓
Online Shrink	✗	✗	✓	✗	✗
Add/Remove Disks	✗	✗	✓	✓	✗

also see: https://en.wikipedia.org/wiki/Comparison_of_file_systems

Credit: Linux file system comparison table created by @dan_nanni on Instagram

Unmounting/Remounting Example

```
hayral@Computer1:~$ mount
```

-
-
-

```
configfs on /sys/kernel/config type configfs (rw,nosuid,nodev,noexec,relatime)
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,relatime)
/dev/sda1 on /boot/efi type vfat (rw,relatime,fmask=0077,dmask=0077,codepage=437,iocharset=iso8859-1,
shortname=mixed,errors=remount-ro)
tmpfs on /run/user/1000 type tmpfs (rw,nosuid,nodev,relatime,size=74660k,mode=700,uid=1000,gid=1000)
gvfsd-fuse on /run/user/1000/gvfs type fuse.gvfsd-fuse (rw,nosuid,nodev,relatime,user_id=1000,group_i
d=1000)
/dev/sr0 on /media/hayral/VBox_GAs_6.1.18 type iso9660 (ro,nosuid,nodev,relatime,nojoliet,check=s,map
=n,blocksize=2048,uid=1000,gid=1000,dmode=500,fmode=400,uhelper=udisks2)
hayral@Computer1:~$ umount /dev/sr0
hayral@Computer1:~$ sudo mkdir /mnt/cdrom
[sudo] password for hayral:
hayral@Computer1:~$ sudo mount -t iso9660 /dev/sr0 /mnt/cdrom
mount: /mnt/cdrom: WARNING: device write-protected, mounted read-only.
hayral@Computer1:~$
```

Determining Device Names

- First, let's look at how the system names devices. If we list the contents of the /dev directory (where all devices live), we can see that there are lots of devices:



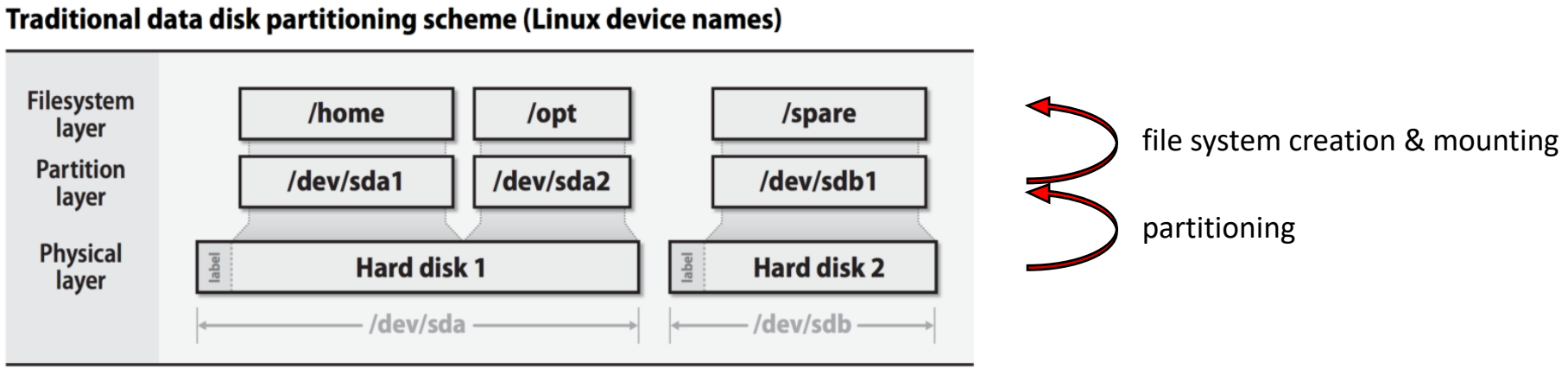
- The contents of this listing reveal some patterns of device naming. Table 15-2 lists a few:

Table 15-2: Linux Storage Device Names

Pattern	Device
*/dev/fd**	Floppy disk drives
*/dev/hd**	IDE (PATA) disks on older systems. Typical motherboards contain two IDE connectors, or *channels*, each with a cable with two attachment points for drives. The first drive on the cable is called the *master* device and the second is called the *slave* device. The device names are ordered such that */dev/hda* refers to the master device on the first channel, */dev/hdb* is the slave device on the first channel; */dev/hdc*, the master device on the second channel, and so on. A trailing digit indicates the partition number on the device. For example, */dev/hda1* refers to the first partition on the first hard drive on the system while */dev/hda* refers to the entire drive.
*/dev/lp**	Printers
*/dev/sd**	SCSI disks. On recent Linux systems, the kernel treats all disk-like devices (including PATA/SATA hard disks, flash drives, and USB mass storage devices such as portable music players and digital cameras) as SCSI disks. The rest of the naming system is similar to the older */dev/hd** naming scheme described above.
/dev/sr*	Optical drives (CD/DVD readers and burners)

Storage Device Names in Linux

- In Linux storage devices like mechanical hard drives and solid state drives are represented as a file named as **/dev/sda**, **/dev/sdb**, **/dev/sdc**, … in order. These are also called block storage devices (unlike the character devices like modems, tty, keyboard and screen)
- Then each partition on each of these devices are further represented as files with the same name suffixed with a number for partition no. (i.e. **/dev/sda1**, **/dev/sda2**, …, **/dev/sdb1**, **/dev/sdb2**, …)
  - These partitions are also block storage devices.
  - You can create a file system on and mount the whole device or a partition of it in the very same way. (Creating a partition, even if there is only one, is the more common way)
  - You can mount a block device to anywhere you want on your root file system.



Creating New Filesystems

- Let's say that we want to reformat the flash drive with a Linux native filesystem, rather than the FAT32 system it has now.
- This involves two steps:
  1. (optional) create a new partition layout if the existing one is not to your liking
  2. creating a new, empty filesystem on the drive.

Warning:

Creating a new file system on a device or partition

is equivalent to formatting it,

your data will be unretrievably lost!

Manipulating Partitions with fdisk

- The **fdisk** program allows us to interact directly with disk-like devices (such as hard disk drives and flash drives) at a very low level.
- With this tool we can edit, delete, and create partitions on the device.
- To work with our drive, we must first **unmount** it (if needed) and then invoke the **fdisk** program as follows:
  - You can also check available commands by pressing **m** and ENTER.
- GPT is the most commonly used partition table type
  - DOS is an older partition type but still used on older systems.

```
hayral@Computer1:~$ sudo umount /dev/sdb1
hayral@Computer1:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.34).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): m

Help:

  GPT
   M   enter protective/hybrid MBR

  Generic
   d   delete a partition
   F   list free unpartitioned space
   l   list known partition types
   n   add a new partition
   p   print the partition table
   t   change a partition type
   v   verify the partition table
   i   print information about a partition

  Misc
   m   print this menu
   x   extra functionality (experts only)

  Script
   I   load disk layout from sfdisk script file
   O   dump disk layout to sfdisk script file

  Save & Exit
   w   write table to disk and exit
   q   quit without saving changes

  Create a new label
   g   create a new empty GPT partition table
   G   create a new empty SGI (IRIX) partition table
   o   create a new empty DOS partition table
   s   create a new empty Sun partition table


Command (m for help):
```

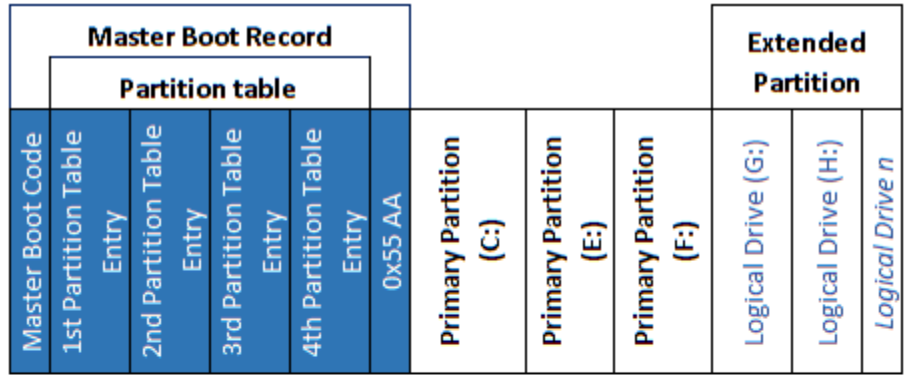
GPT vs MBR Partition Table

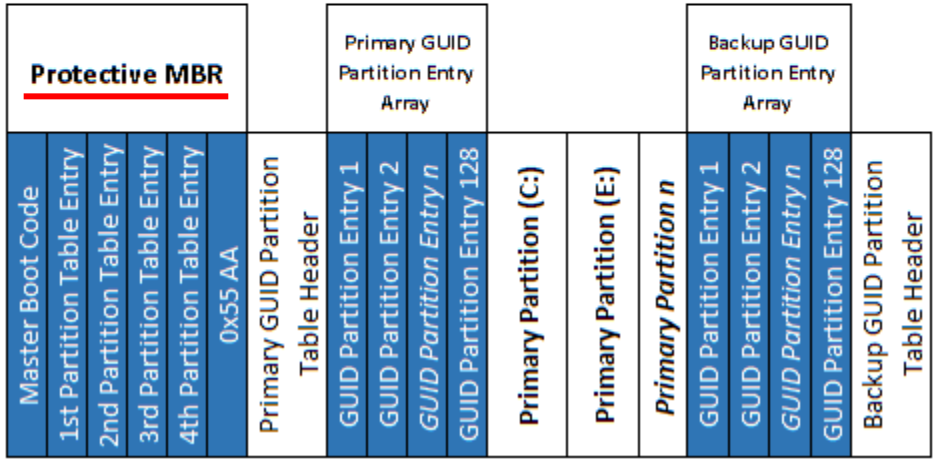
	MBR (Master Boot Record)	GPT (GUID Partition Table)
Maximum Supported Disk Size	2TB	9.4ZB (1ZB=10^9 TB)
Maximum Number of Partitions	4 primary (or 3 primary + many logical)	128 primary
Firmware Interface	BIOS	UEFI
Released	Early 1980s	Specified in 1990, wide use in 2000s

Demo Here:

1. Create partition table
2. Create partition
3. Create file system
4. Mount

MBR





GPT

Creating a New Filesystem with mkfs

- With our partition editing done, it's time to create a new filesystem on our drive.
- To do this, we will use **mkfs** (short for make filesystem), which can create filesystems in a variety of formats.
- To create an **ext3** filesystem on the device, we use the **-t** option to specify the **ext3** system type, followed by the name of the device containing the partition we wish to format:

```
hayral@Computer1:~$ sudo mkfs -t ext3 /dev/sdb1
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 25339 4k blocks and 25344 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done

hayral@Computer1:~$
```

mkfs

- As a matter of fact **mkfs** itself does not contain any code for file system creation.
- Based on the file system type you specify it searches the system for the actual binary which will create the file system.
- You can check which file systems **mkfs** can create for you.

```
hayral@Computer1:~$ ls /usr/sbin/mkfs.*
/usr/sbin/mkfs.bfs     /usr/sbin/mkfs.ext4     /usr/sbin/mkfs.msdos
/usr/sbin/mkfs.btrfs   /usr/sbin/mkfs.fat      /usr/sbin/mkfs.ntfs
/usr/sbin/mkfs.cramfs  /usr/sbin/mkfs.hfs      /usr/sbin/mkfs.reiserfs
/usr/sbin/mkfs.exfat   /usr/sbin/mkfs.hfsplus  /usr/sbin/mkfs.vfat
/usr/sbin/mkfs.ext2    /usr/sbin/mkfs.jfs      /usr/sbin/mkfs.xfs
/usr/sbin/mkfs.ext3    /usr/sbin/mkfs.minix
hayral@Computer1:~$
```

- In order to use a new file system, you must also have the necessary file system driver on your kernel.
- When you install a new file system to linux, it installs the necessary **kernel driver** and also creates the necessary **mkfs.*** ** file system creation tool and the specific **fsck** binary.

Testing and Repairing Filesystems

- In our earlier discussion of the ***/etc/fstab*** file, we saw some digits at the end of each line.
- Each time the system boots, it checks the integrity of the filesystems before mounting them.
- This is done by the **fsck** program (short for *filesystem check*).
- The last number in each ***fstab*** entry specifies the order in which the devices are to be checked.
  - The **root filesystem** is checked first, followed by the ***home*** and ***boot*** filesystems.
  - Devices with a zero as the last digit are **not** routinely checked.
- In addition to checking the integrity of filesystems, **fsck** can also repair corrupt filesystems with varying degrees of success, depending on the amount of damage.
- On Unix-like filesystems, recovered portions of files are placed in the ***lost+found*** directory, located in the root of each filesystem.

Testing and Repairing Filesystems

- To check our drive (which should be unmounted first), we can do the following:

```
hayral@Computer1:~$ sudo fsck /dev/sdb1
fsck from util-linux 2.34
e2fsck 1.45.5 (07-Jan-2020)
/dev/sdb1: clean, 11/25344 files, 1833/25339 blocks
hayral@Computer1:~$
```

- Works directly on data structures of file system stored on disk, which are internal and specific to the particular file system in use - so an fsck for that specific file system is required.
- **The file system is normally checked while unmounted, mounted read-only, or with the system in a special maintenance mode. (see next slide)**
- Full copy-on-write (**CoW**) file systems such as **ZFS** and **Btrfs** are designed to avoid most causes of corruption and have no traditional "fsck" repair tool.
  - Both have a "scrub" utility which examines and repairs any problems; in the background and on a mounted file system.

WHAT THE FSCK?

In Unix culture, fsck is often used in place of a popular word with which it shares three letters. This is especially appropriate, given that you will probably be uttering the aforementioned word if you find yourself in a situation where you are forced to run fsck.

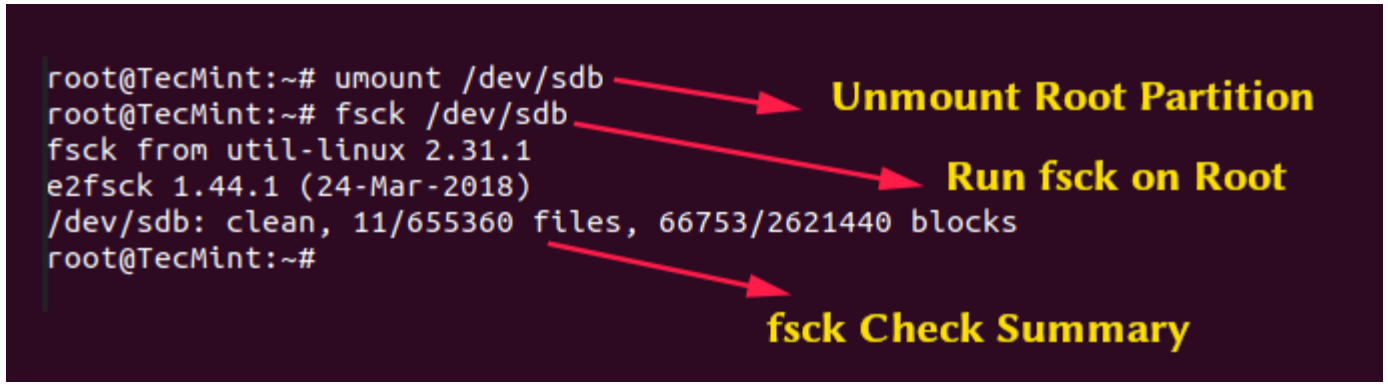
fsck

Do not run fsck on a live or mounted file system !

- Running **fsck** on a mounted filesystem can usually result in disk and/or data corruption.
- This behavior is different from Windows, where you can run disk check and repair file system errors on a live system.
- Linux doesn't let you use fsck on a mounted file system anyway:

```
hayral@Computer1:~$ sudo fsck /dev/sdb1
fsck from util-linux 2.34
e2fsck 1.45.5 (07-Jan-2020)
/dev/sdb1 is mounted.
e2fsck: Cannot continue, aborting.

hayral@Computer1:~$
```



Moving Data Directly to and from Devices

- Data on our computers as being organized into files, but it is also possible to think of the **data in "raw" form**.
- disk drive → consists of a large number of "blocks" of data
  - OS sees those as directories and files through a **file system driver**.
- We can treat a disk drive as a large collection of data blocks and we can perform useful tasks, such as cloning devices.

dd command in Linux – copy data:

- dd if=<input_file> of=<output_file>: Copies data from an input file to an output file with specified block sizes and counts.
- dd if=/dev/zero of=<output_file> bs=<block_size> count=<count>: Generates a file filled with zeros of a specified size.

```
dd if=input_file of=output_file [bs=block_size [count=blocks]]
```

These can be real files or block devices

Warning: *The dd command is very powerful. Though its name derives from* data definition, *it is sometimes called* destroy disk *because users often mistype either the* `if` *or* `of` *specifications. Always double-check your input and output specifications before pressing* ENTER*!*

md5sum

- It’s often useful to verify the integrity of a file that we have downloaded.
- In some cases, the distributor of a file will also supply a checksum value or a checksum file.
- A checksum is the result of a mathematical calculation resulting in a number that represents the content of the target file. (think like a finger print of the file)
- If the contents of the file change by even one bit, the resulting checksum will be totally different.
- The most common method of checksum generation uses the **md5sum** program.
- When you use md5sum, it produces a unique hexadecimal number:

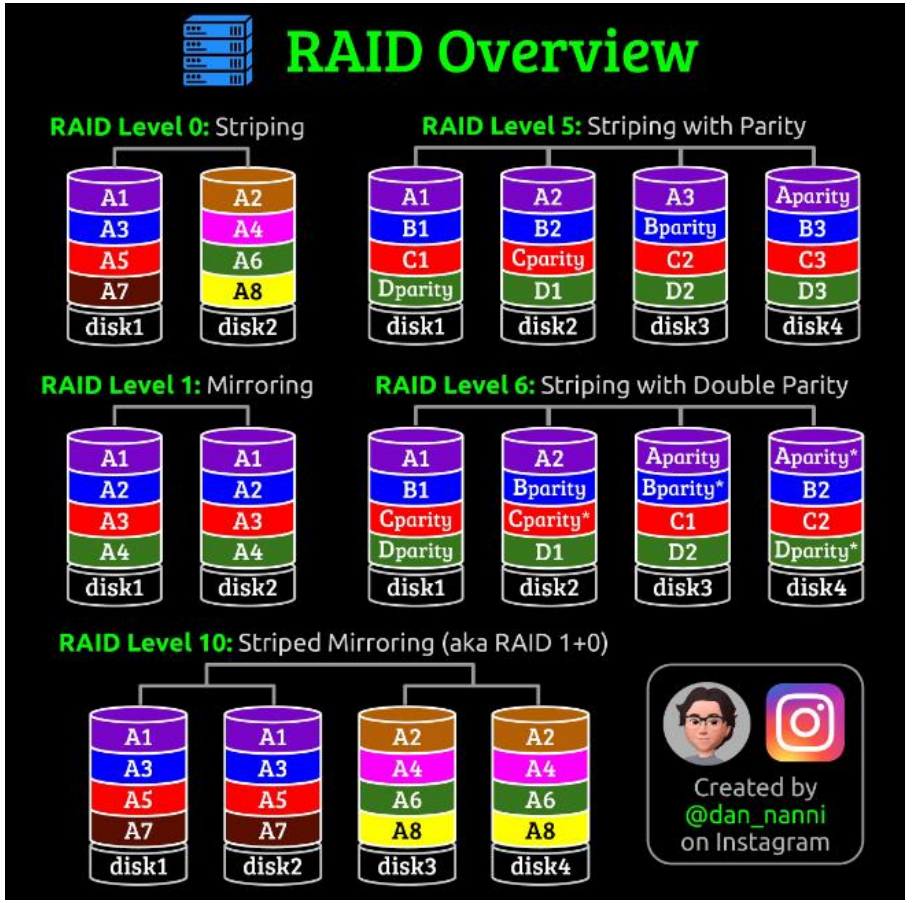
```
hayral@Computer1:~$ md5sum DownloadedFile.zip
f1b42f8f5603bcd99d3fbd4c5b587adb  DownloadedFile.zip
hayral@Computer1:~$
```

*<u>Note:</u> Today few people still use md5, most common hash tool today is SHA256 (**sha256sum** command); for example Ubuntu stopped providing md5 values of their downloadable files for verification anymore.*

RAID

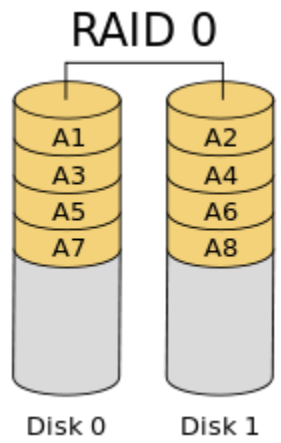
("Redundant Array of Inexpensive Disks" or "Redundant Array of Independent Disks")

- **RAID** is a data storage virtualization technology that **combines multiple physical disk drive components into one or more logical units** for the purposes of data redundancy, performance improvement, or both.
- This was in contrast to the previous concept of highly reliable mainframe disk drives referred to as "single large expensive disk" (**SLED**).
- Data is distributed across the drives in one of several ways, referred to as RAID levels, depending on the required level of redundancy and performance.
- Different layouts are named "RAID" followed by a number, for example RAID **0** or RAID **1**.
- Each RAID level, provides a different balance among the key goals: **reliability**, **availability**, **performance**, and **capacity**.
- RAID levels greater than RAID 0 provide **protection against unrecoverable sector read errors, as well as against failures of whole physical drives**.



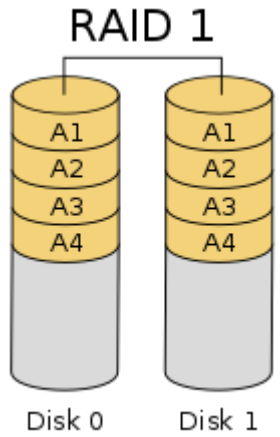
RAID 0 - Striping

- RAID 0 (*a.k.a stripe set or striped volume*) splits ("stripes") data evenly across two or more disks, without parity information, redundancy, or fault tolerance.
- Since RAID 0 provides **no fault tolerance or redundancy**, the failure of one drive will cause the entire array to fail → total data loss
  - This configuration is typically implemented having speed as the intended goal.
- RAID 0 increase **performance** or just a way to create a **large logical volume** out of multiple physical disks.
- RAID 0 setup can be created with disks of different sizes
  - Storage space added to the array by each disk is limited to the size of the smallest disk.
  - For example, if a 120 GB disk is striped together with a 320 GB disk, the size of the array will be 120 GB × 2 = 240 GB.
    - some RAID implementations allow the remaining 200 GB to be used for other purposes
- Figure shows how the data is distributed into Ax stripes on two disks, A1:A2 → first stripe, A3:A4 → second stripe, ...
- **Stripe size** is defined during the creation of a RAID 0 array, can't change later.
- Stripes are accessed in parallel. n-drive RAID 0 array appears as a single large disk with a data rate n times higher than the single-disk rate.



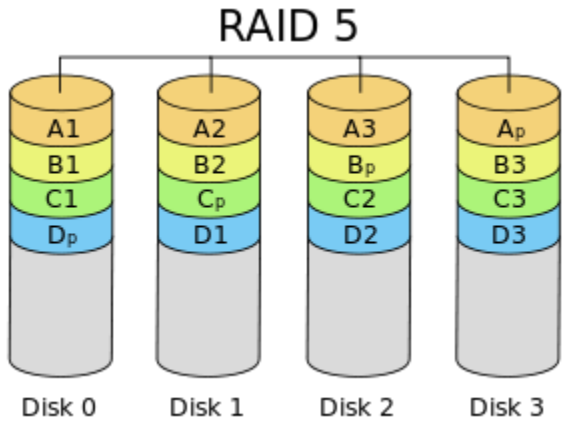
RAID 1 - Mirroring

- RAID 1 consists of an exact copy (or mirror) of a set of data on two or more disks; a classic mirrored pair contains two disks.
- This configuration offers **no parity**, **no striping**, and **no spanning of disk space** across multiple disks
  - data is mirrored on all disks belonging to the array
  - array can only be as big as the smallest disk
- Useful when read performance or reliability is more important than write performance and storage capacity.
- Array will continue to operate as long as **at least one member drive is operational**.
- **Performance**
  - Any read request can be serviced and handled by any drive in the array; thus, depending on the nature of I/O load, random read performance of a RAID 1 array may equal up to the **sum of each member's performance**, while the write performance remains at the level of a **single disk**. However, if disks with different speeds are used in a RAID 1 array, overall write performance is equal to the **speed of the slowest disk**.
  - Synthetic benchmarks show performance improvements when multiple HDDs or SSDs are used in a RAID 1 setup, compared to single-drive performance.



RAID 5 – Striping+Parity

- RAID 5 : block-level striping with distributed parity.
  - parity information is distributed among the drives
- Requires that all drives but one to be present to operate.
- Upon failure of a single drive, subsequent reads can be calculated from the distributed parity such that **no data is lost**.
- RAID 5 requires at least three disks.
- Many layouts of data and parity in a RAID 5 disk drive array depending upon the sequence of writing across the disks, that is:
  1. sequence of data blocks written, left to right or right to left on the disk array, of disks 0 to N
  2. location of the parity block (beginning or end of the stripe)
  3. location of the first block of a stripe with respect to parity of the previous stripe.
- The figure to the right shows 1) data blocks written **left to right**, 2) the **parity block at the end** of the stripe and 3) the **first block of the next stripe not on the same disk as the parity block of the previous stripe**.
- Since parity calculation is performed on the full stripe, small changes to the array experience **write amplification**
  - worst case : when a single, logical sector is to be written, original sector and the corresponding parity sector is read, original data is removed from the parity, new data calculated into the parity and both the new data sector and the new parity sector are written.



File system commands

Common	**mkfs** formats a partition or a device with a specific filesystem **mkswap** creates a Linux swap area on a device or in a file **df** shows disk space used and available on mounted filesystems **du** shows space used by individual files and directories **lsblk** shows info about block devices, e.g. size, filesystem type **file** identifies a file's type independent of its extension **fdisk/cfdisk** creates and manages partitions on disks **mount/umount** mounts or unmounts a filesystem **lsof** displays info about files opened by processes **stat** shows detailed attributes of a file or a filesystem **smartctl** monitors the health of SATA/SCSI devices
EXT	**e2fsck** checks and optionally repairs inconsistencies in filesystem **resize2fs** extends or shrinks a filesystem on the fly **tune2fs** modifies filesystem parameters for performance/features **dumpe2fs** displays detailed info of a filesystem's metadata **lsattr/chattr** lists or modifies extended attributes of files
ZFS	**zpool** manages ZFS storage pools for redundancy and snapshots **zfs** creates, destroys, lists and modifies filesystem in ZFS pools **zdb** troubleshoots ZFS storage pools and datasets
Btrfs	**btrfs** swiss army knife for managing Btrfs filesystems **btrfsck** checks the integrity of Btrfs filesystems and repair them
XFS	**xfs_repair** analyzes and fixes inconsistencies in filesystem **xfs_growfs** expands the size of a mounted filesystem **xfs_admin** modifies various properties of unmounted filesystem **xfs_info** shows detailed info on mounted/unmounted filesystem **xfsdump/xfsrestore** handles backups/restorations of filesystem